ASP.NET MVC

Mme. Fatima-Ezzahra AIT BENNACER
f.aitbennacer@emsi.ma

2026 - 2027

Plan du cours

1. Introduction à .Net
2. L'architecture .Net
3. Les bases .Net
4. ASP.Net
5. Modèle MVC
6. ASP.Net & MVC
7. Création d'une page web avec ASP.Net

Introduction à .Net

C'est quoi .NET ?

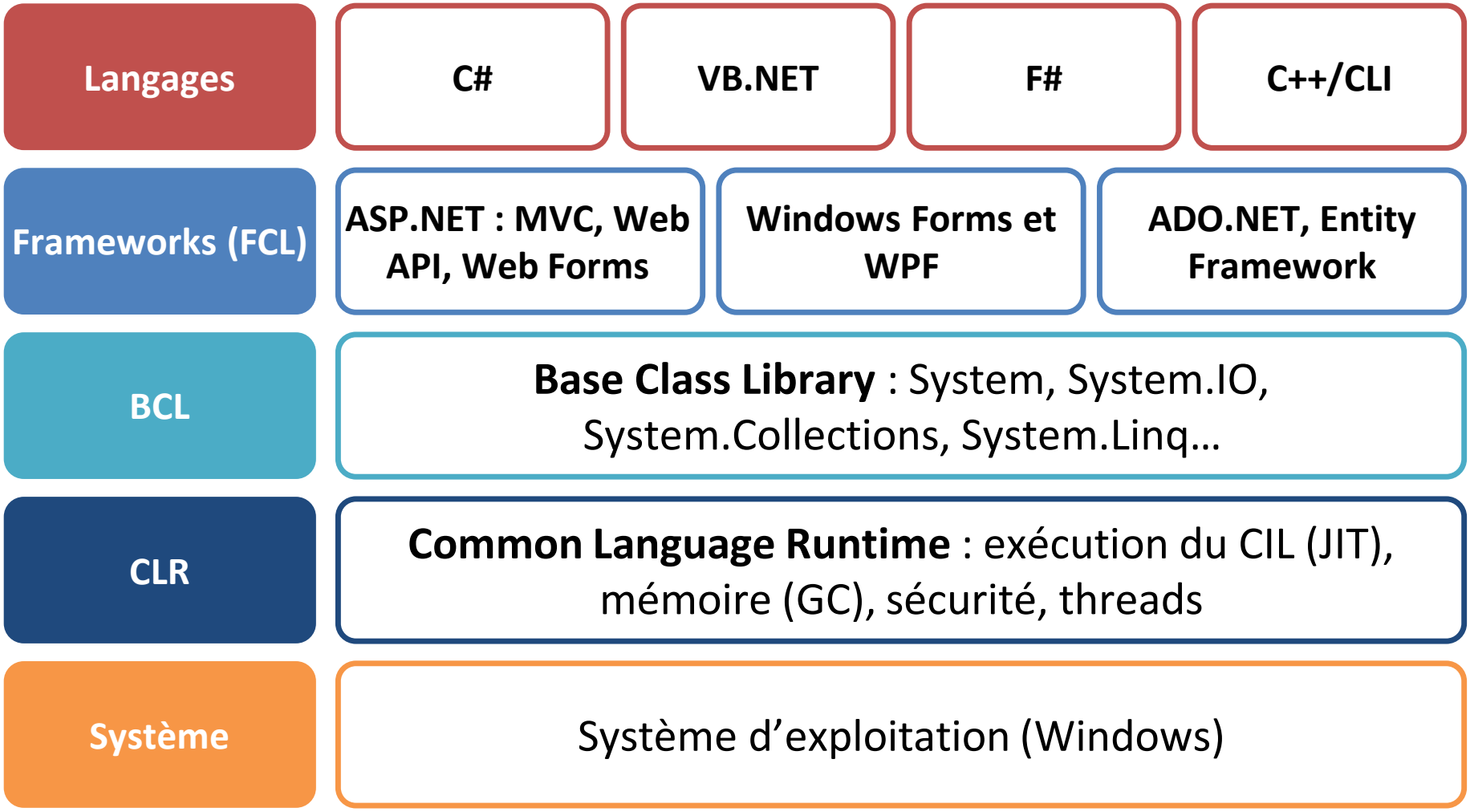
.NET est la plateforme de développement de **Microsoft** : un ensemble d'outils, d'un environnement d'exécution et d'une très vaste bibliothèque de classes pour créer des applications web, de bureau, mobiles ou des services.

Sa première version, le .NET Framework 1.0, date de 2002.



Architecture de .NET Framework

Architecture .NET



Architecture .NET

- **Le CLR (Common language Runtime)** :
  - La machine virtuelle de .NET (comme la JVM de Java)
  - Compile le CIL (code intermédiaire) en code natif à l'exécution JIT (Just-In-Time)
  - Ramasse-miettes (Garbage Collector), sécurité, types, exceptions, threads
- **La BCL (Base Class Library):** Les classes de base de .NET:
  - Les classes de base de tous les programmes
  - Chaînes, collections, fichiers, dates…
  - System, System.IO, System.Linq…

Architecture .NET

- **FCL (Framework Class Library)** :
  - Plus large que la BCL : elle la contient
  - Ajoute les frameworks applicatifs
  - ASP.NET, Windows Forms, ADO.NET…

- **Au sommet de la pile**, nous avons les langages supportés par .NET tels que **C#, VB.NET, C++, F#**, qui sont des produits de Microsoft, et d'autres tels que **Cobol, Delphi**, etc.

Les bases .Net

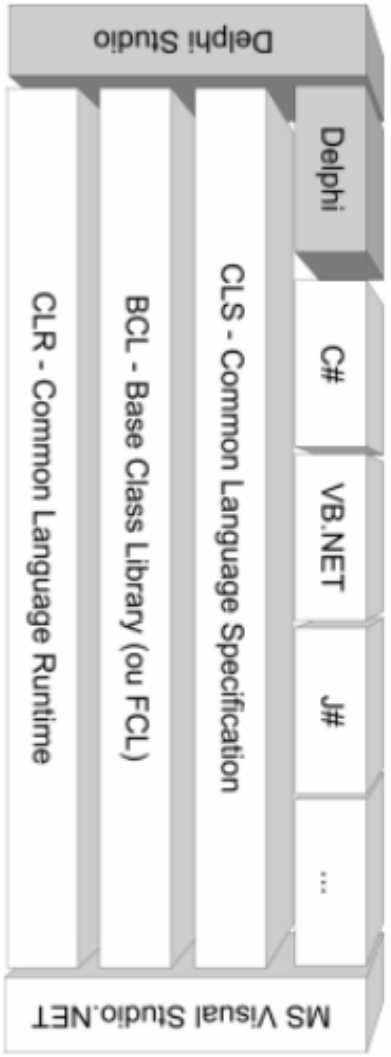
Les bases .NET

- **Le CLR**

  Ce runtime est le moteur de .NET, c'est lui qui est en charge de l'exécution des logiciels écrits pour .NET compilés en CIL (Common Intermediate Language) avec un système appelé JIT (Just in Time).

  Le CLR gère:

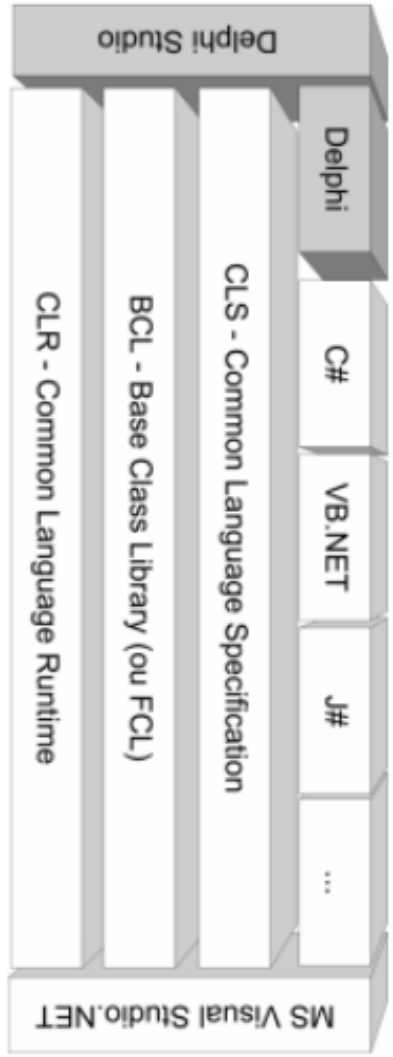
  - La compilation en code natif et l'exécution de code IL (Intermediate Language).
  - La gestion de la sécurité.
  - La gestion de la mémoire.
  - La gestion des processus.
  - La gestion des threads (tâches).



Les bases .NET

- **La BCL (Base Class Library)**

  - Se plaçant directement au dessus du CLR
  - Offre de nombreuses classes, interfaces et types qui forment le socle des développements sous .NET
  - Permet d'unifier les développements puisqu'elle fournit l'ensemble des outils de base aux applications



Les bases .NET

L'espace de noms et assembly

Espace de noms (namespace)

Nature : Regroupement logique de classes

Rôle : Organiser le code, éviter les conflits de noms

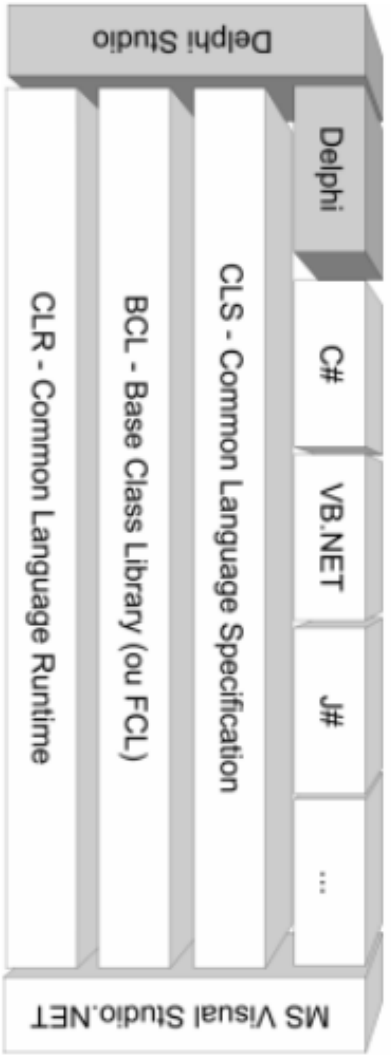
Exemple : `System.Data.SqlClient`

Assembly (assemblée)

Nature : Fichier physique : .dll ou .exe

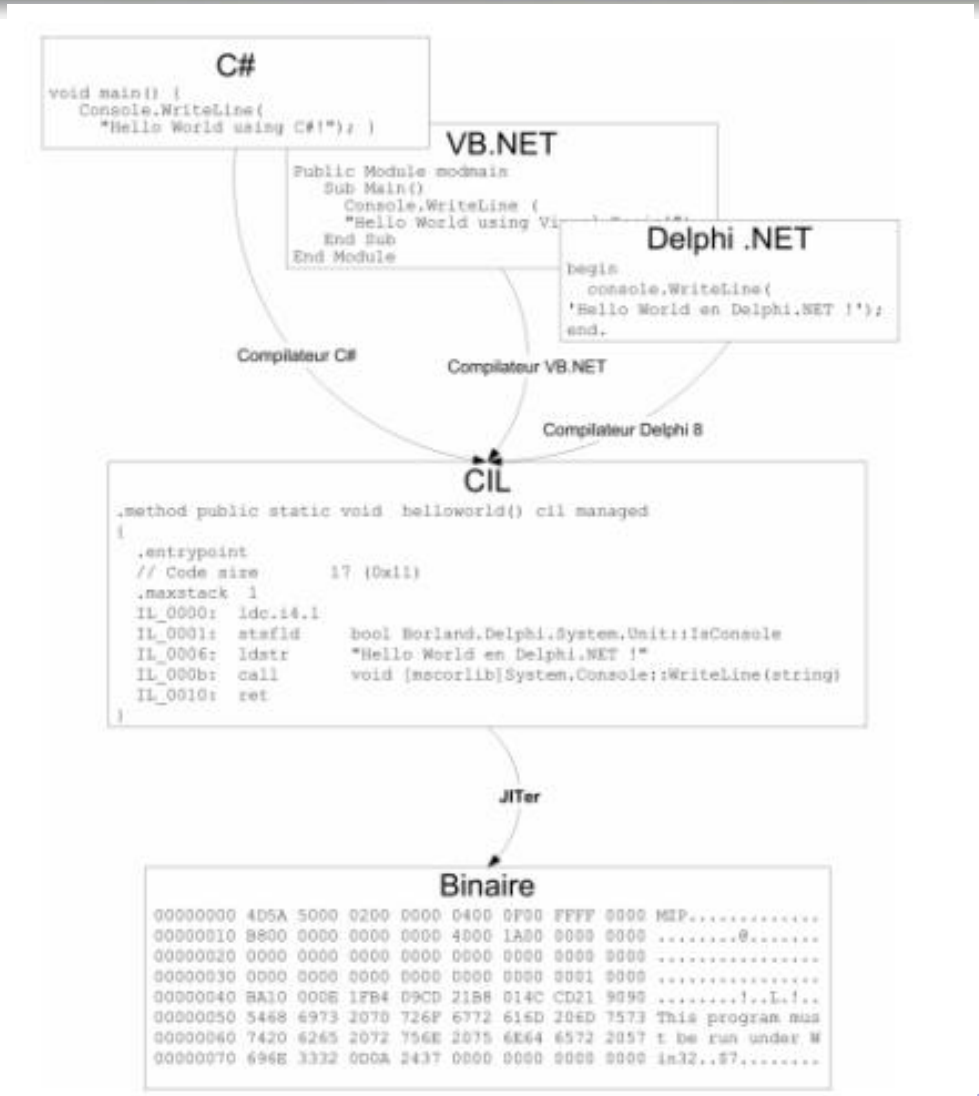
Rôle : Compiler, déployer et protéger (internal)

Exemple : `System.Data.dll`

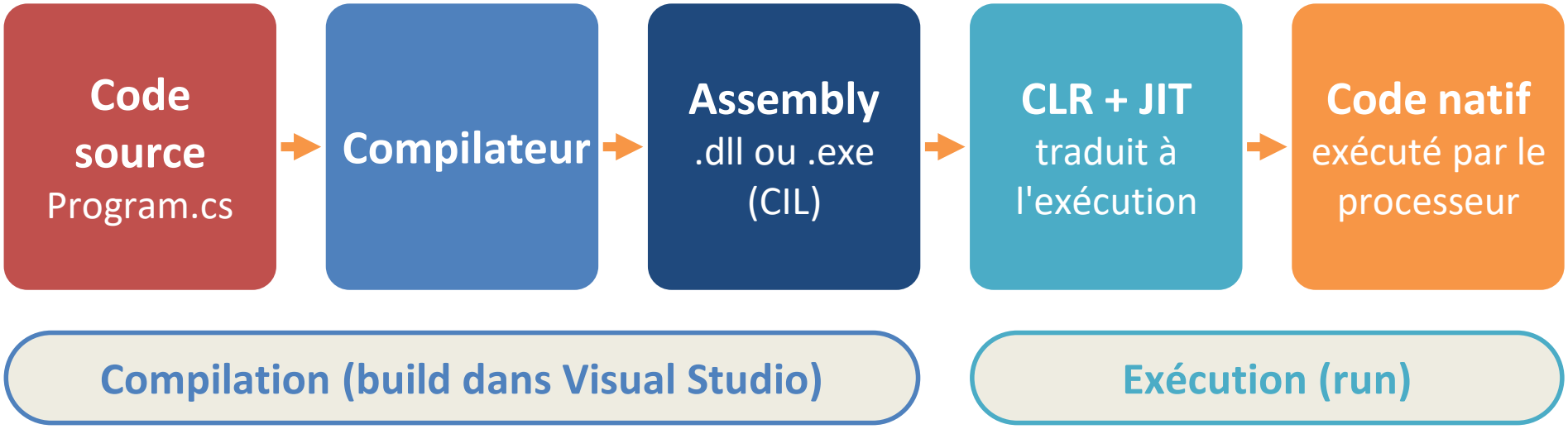


- *Un assembly peut contenir plusieurs espaces de noms, et un espace de noms peut être réparti sur plusieurs assemblies.*

Du code source au binaire exécuté



Du code source au binaire exécuté



ASP .Net

C'est quoi ASP.NET

- Plateforme de développement d'applications web sous Windows
- Utilise, par défaut, le serveur Web de référence de Microsoft: IIS (pour Internet Information Services)
- Repose sur le Framework classique .NET
- Plateforme Web unifiée fournissant les services nécessaires à la création des applications
- Ne dépend ni du langage de programmation ni du navigateur

C'est quoi ASP.NET

- La plateforme de Microsoft pour créer des applications et services web.

Web Forms

Pages à contrôles serveur, programmation par événements

Applications historiques

MVC

Séparation Modèle - Vue - Contrôleur, contrôle total du HTML et des URL

Applications web dynamiques (ce cours)

Web API

Services HTTP qui renvoient des données (JSON, XML)

Back-end d'applications mobiles ou JavaScript

ASP.Net - Modèle MVC



MVC

- Introduit en 1978 par Trygve Reenskaug.
- Permettant de bien organiser le code source
- Conçu, initialement, pour les applications client-lourd et ensuite généralisé aux applications client-léger (Web)
- Une approche consistant à séparer l'affichage des informations, les actions de l'utilisateur et l'accès aux données
- Chacun de ces composants est construit pour manipuler un aspect particulier de développement de l'application
- Indispensable pour des applications dynamiques et de taille importante

MVC: Les trois composants

1. Modèle: La partie qui concerne les données et l'état de notre application. Pour le cas d'une base de données, on peut utiliser un ORM (Object-Relational Mapping) comme Entity Framework pour ASP.NET MVC

2. Vue: La partie qui concerne l'affichage : l'interface avec laquelle l'utilisateur interagit (HTML + CSS...)

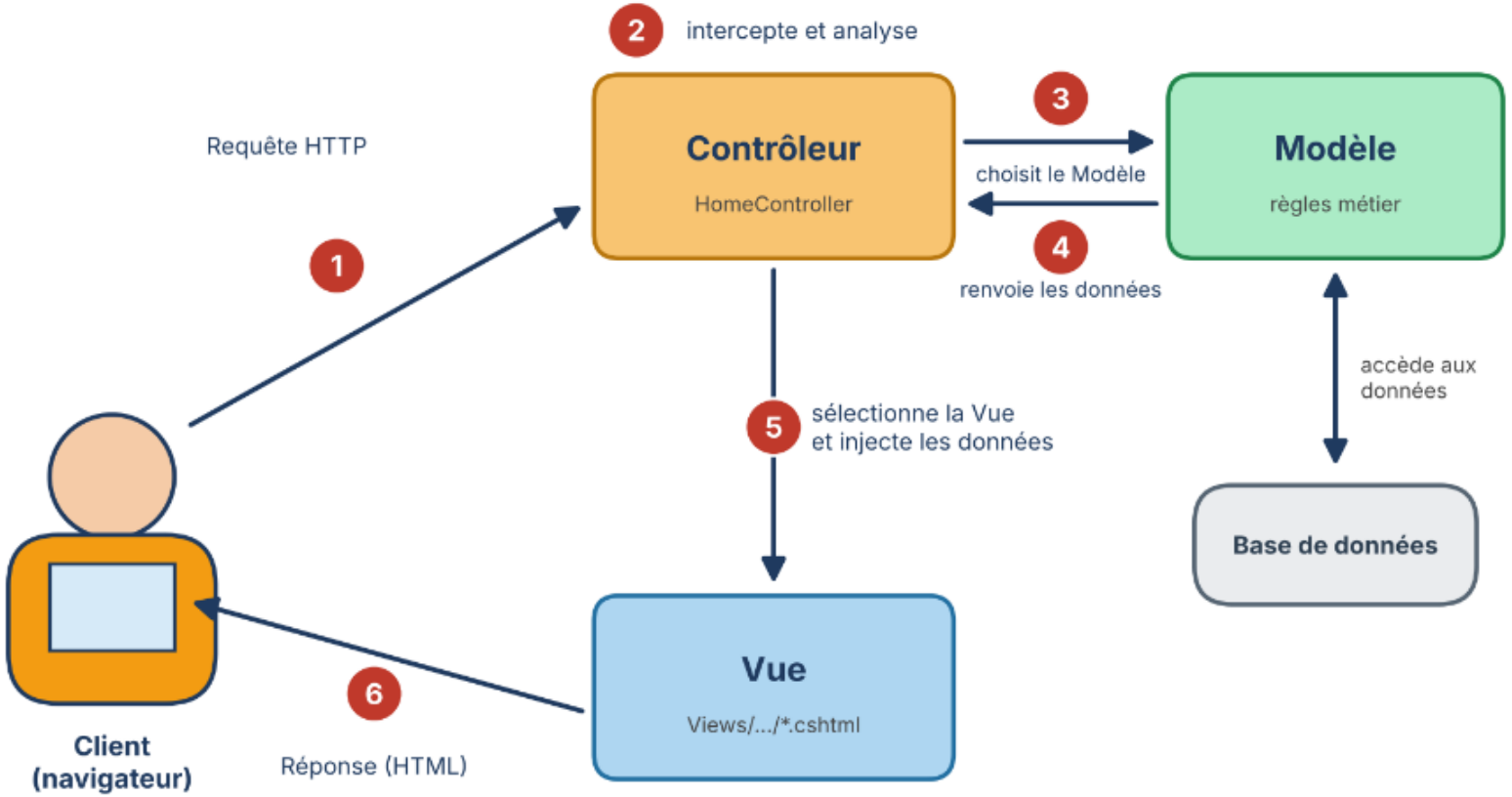
3. Contrôleur: c'est l'intermédiaire entre le modèle et la vue. Il reçoit la requête de l'utilisateur. Il demande les données au modèle, les analyse et renvoie le résultat à afficher à la vue

ASP.NET MVC

- Présenté en 2007 par Scott Guthrie et intégré dans ASP.NET depuis 2009
- Framework de développement d'applications web selon le design pattern: MVC

- Par rapport à ASP.NET:
  - ASP.NET MVC permet de structurer d'avantage l'application, en créant des composants avec des rôles bien identifiés
  - Indispensable pour des applications dynamiques et de taille importante

ASP.NET MVC: Déroulement



Cycle d'une requête MVC

1. Le client envoie une requête depuis la Vue
2. Le Contrôleur intercepte et analyse la requête
3. Il détermine la partie du Modèle concernée
4. Le Modèle gère les données, applique les règles métier et renvoie les données au Contrôleur
5. Le Contrôleur sélectionne la Vue et lui injecte les données
6. La Vue affiche les données au client

ASP.NET MVC: Déroulement

- MVC est un patron de conception particulièrement adapté pour réaliser une application web

- MVC est l'acronyme de Modèle-Vue-Contrôleur :
  1. Le Modèle contient les données de l'application
  2. La Vue contient le code pour afficher les pages de l'application
  3. Le Contrôleur gère les interactions de l'utilisateur en faisant le lien avec le Modèle et la Vue

- MVC permet une séparation claire des intentions et optimise la création d'une application web ou d'un site, sa maintenance et ses tests automatisés.

ASP.NET vs ASP.NET Core

ASP.NET

- Pour Windows.
- Utilise le runtime .NET Framework.
- Bonnes performances.
- Supporte C#, F# et VB.
- Disponible sous Visual Studio.

ASP.NET Core

- Pour Windows, Mac et Linux.
- Utilise le runtime .NET Core.
- Plus performant que ASP.NET.
- Supporte C# et F#.
- Disponible sous VS, VSC.

Exemple d'application web, IDE pour ASP.NET MVC

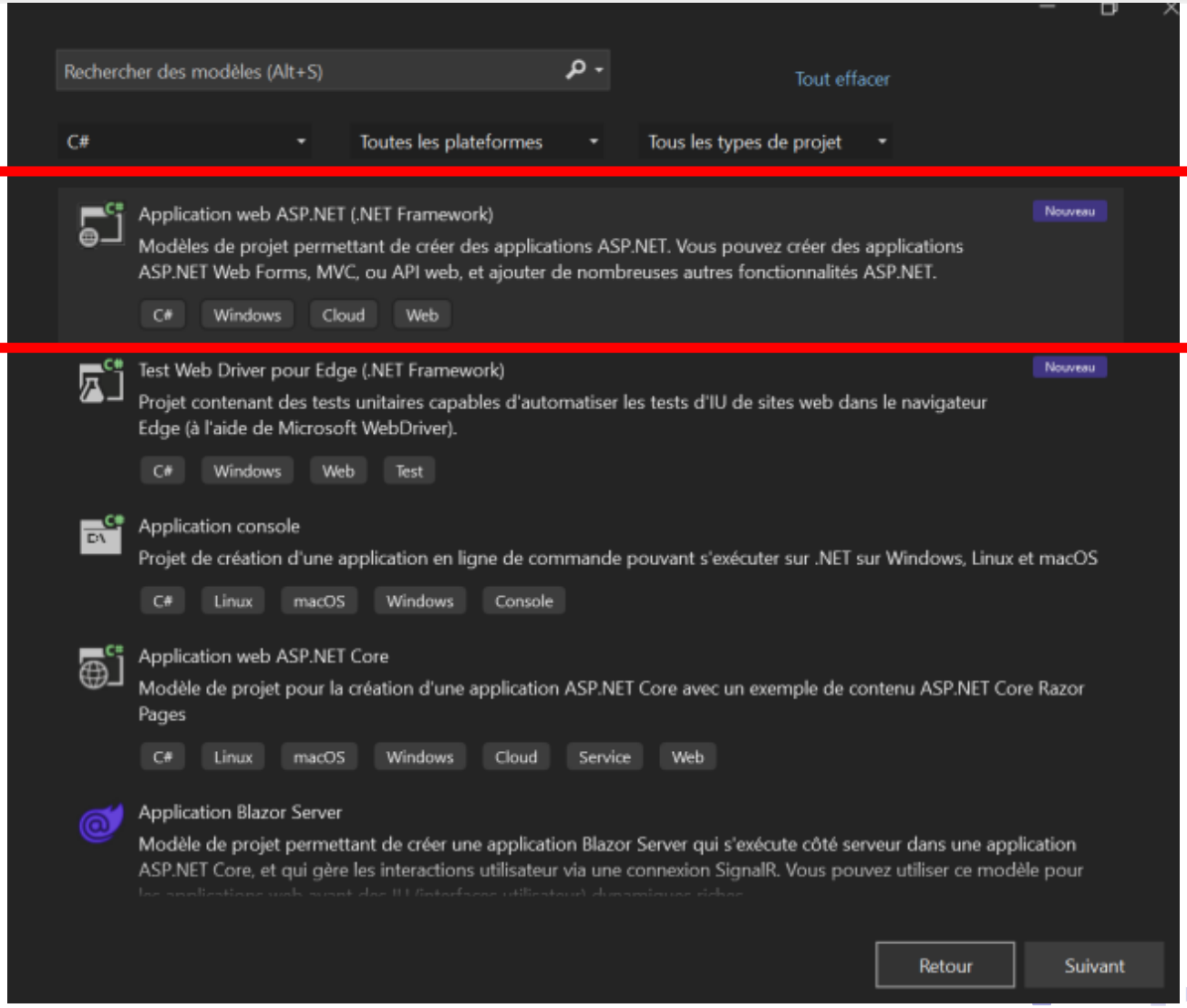
IDE pour ASP.NET MVC

- IDE pour ASP.NET MVC:
  - Visual Studio Community
  - JetBrains Rider
  - Visual Studio Code *(peu adapté pour Windows)*

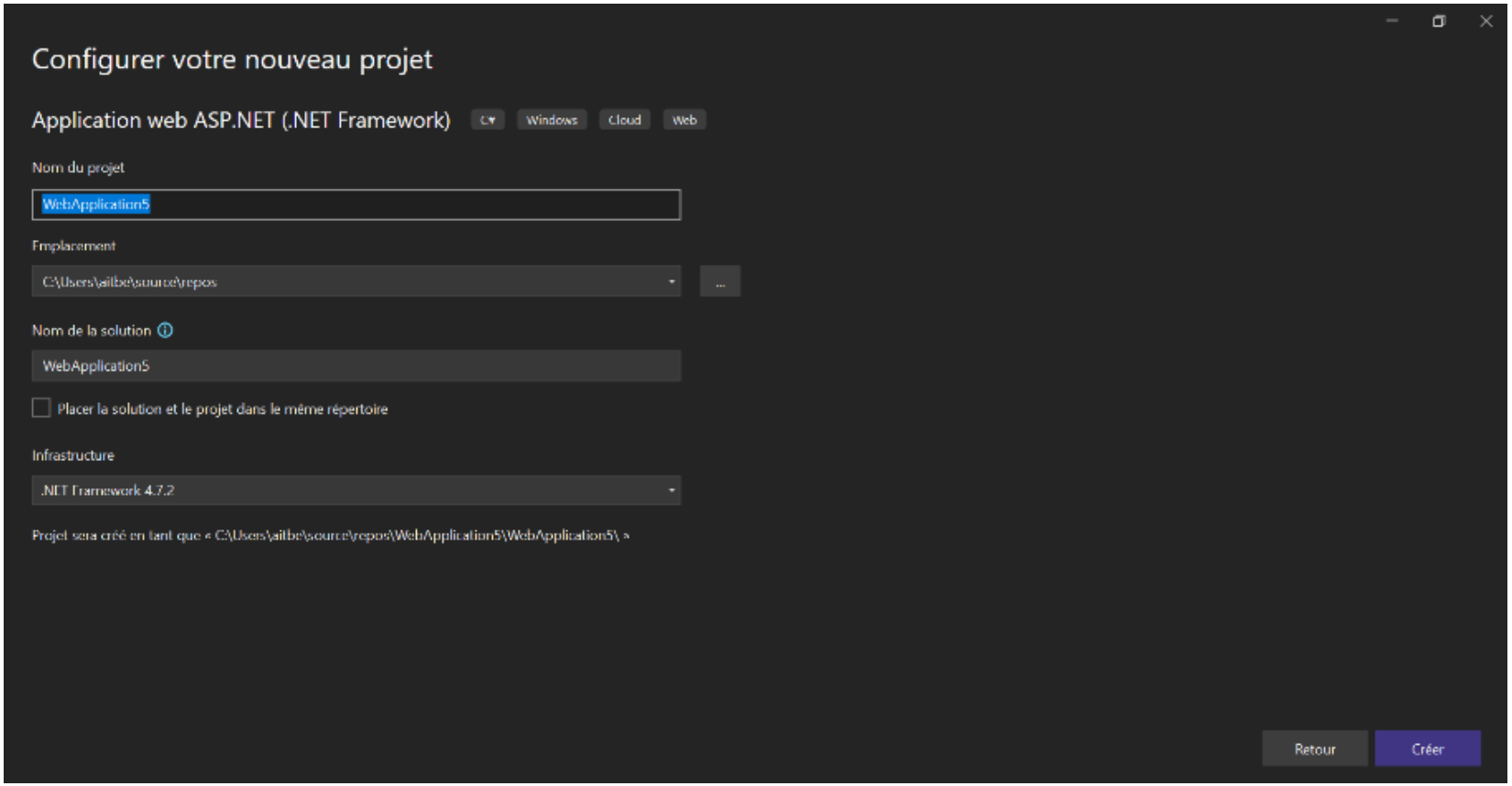
- Installation du .Net Framework: https://dotnet.microsoft.com/en-us/download/dotnet-framework/

  Choisissez developer pack offline installer

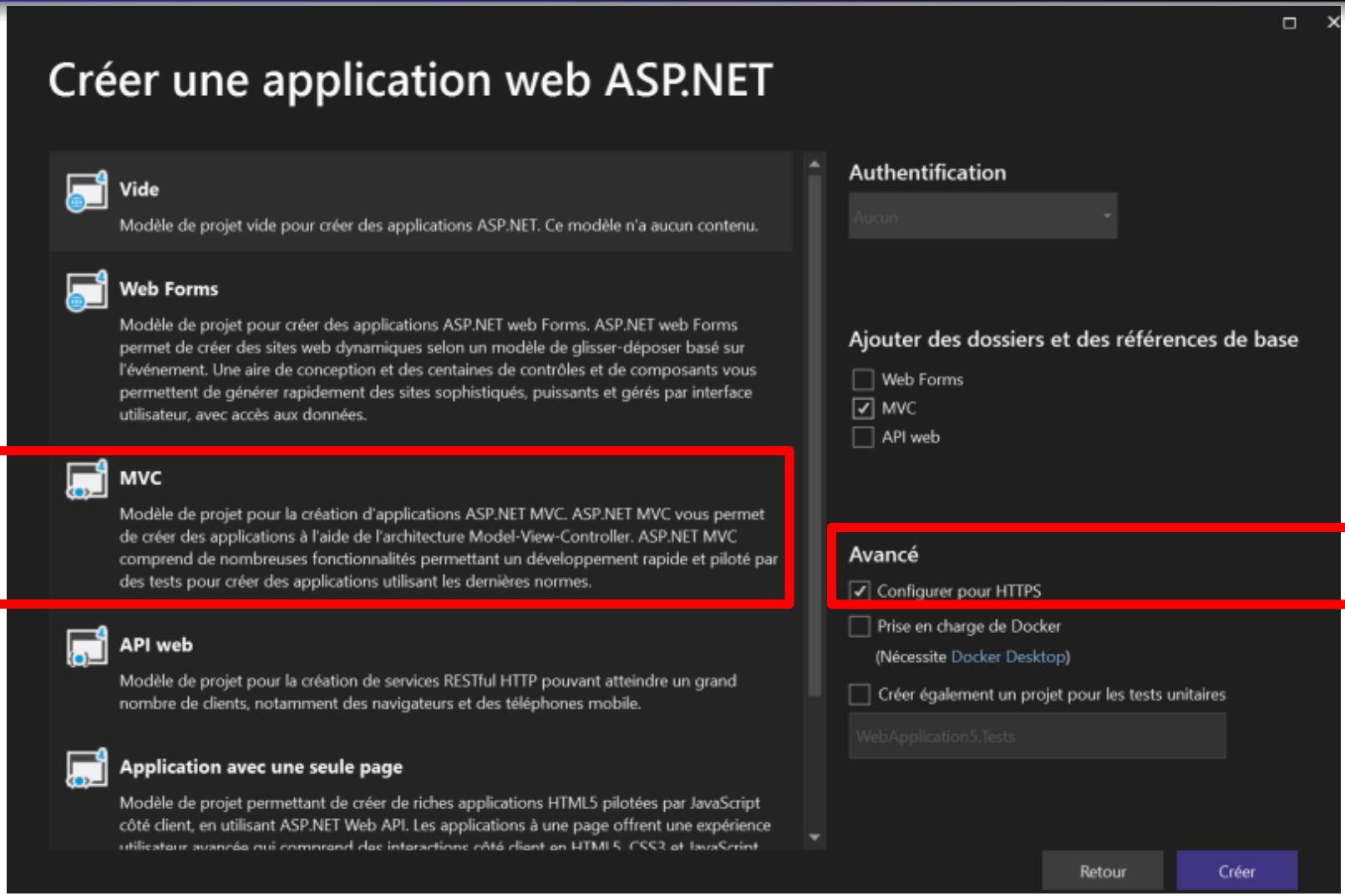
Création d'une page web avec ASP.NET



Création d'une page web avec ASP.NET



Création d'une page web avec ASP.NET



Création d'une page web avec ASP.NET

```
TP_3/
├── App_Start/          (RouteConfig.cs)
├── Content/            (CSS)
├── Controllers/
(HomeController.cs)
├── Models/
├── Scripts/            (JavaScript)
├── Views/
│   ├── Home/           (Index.cshtml)
│   ├── Shared/         (_Layout.cshtml)
│   └── _ViewStart.cshtml
├── Global.asax
├── packages.config
└── Web.config
```

Convention plutôt que configuration

ASP.NET MVC repose sur des conventions de nommage :

HomeController
↓ **URL /Home/…**
Views/Home/

Respectez-les : c'est ce qui permet au framework de retrouver vos classes et vos fichiers.

Rôle des dossiers et fichiers

`Controllers/`
Les classes contrôleurs et leurs actions (C#)

`Models/`
Les classes qui représentent et manipulent les données

`Views/`
Les pages .cshtml, rangées par contrôleur : Views/Home/ pour Home

`_Layout.cshtml`
Le modèle commun à toutes les pages (menu, pied de page)

`App_Start/`
La configuration, dont RouteConfig.cs (routage)

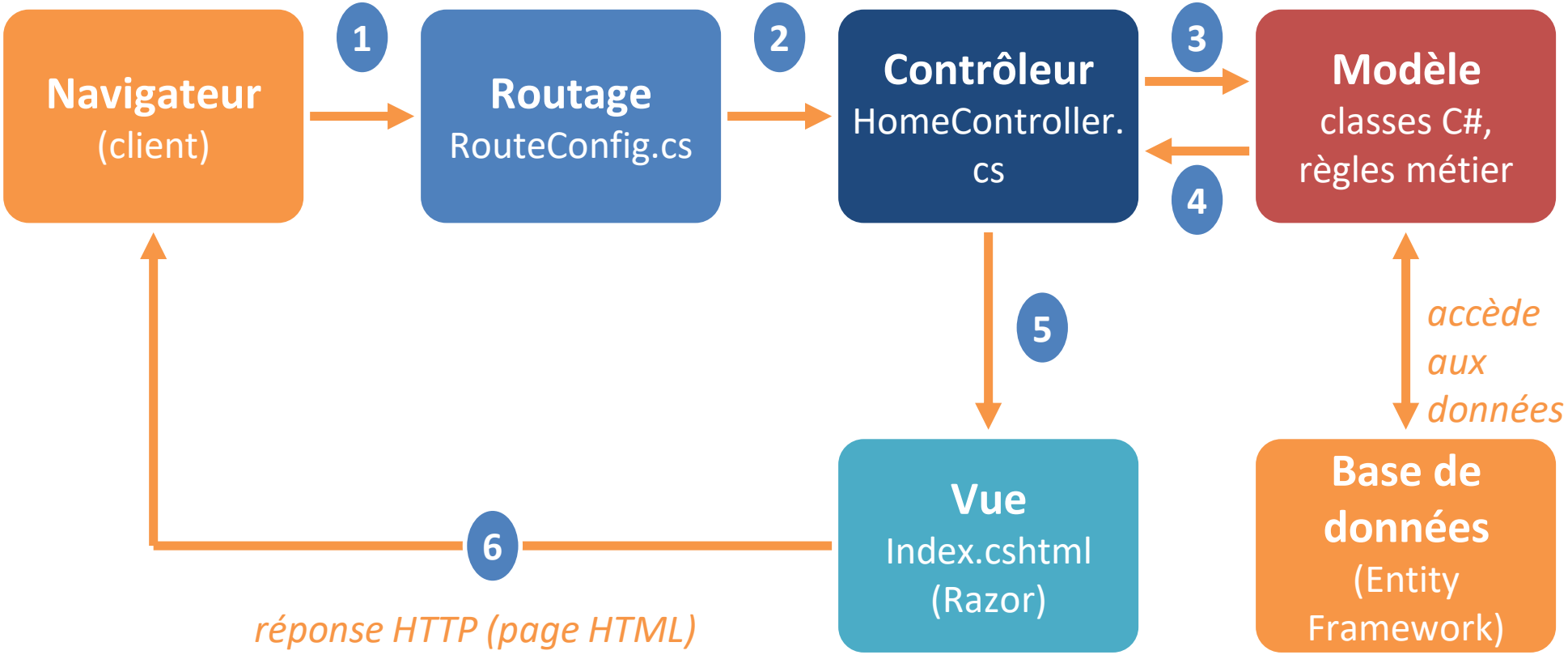
`Global.asax`
Le point de départ : il appelle les classes de App_Start/

`Content/`, `Scripts/`
Les ressources statiques : CSS, JavaScript, images

`packages.config`
La liste des packages NuGet installés

`Web.config`
La configuration (chaînes de connexion, paramètres…)

ASP.NET MVC: Cycle de vie d'une requête



ASP.NET MVC: Cycle de vie d'une requête

1. Le navigateur envoie une requête HTTP (une URL) au serveur.
2. Le routage associe l'URL à un contrôleur, une action et ses paramètres.
3. Le contrôleur exécute l'action et demande au modèle les données ou traitements.
4. Le modèle applique les règles métier, accède à la base et renvoie le résultat au contrôleur.
5. Le contrôleur choisit la vue et lui transmet les données.
6. La vue génère la page HTML, renvoyée au navigateur dans la réponse HTTP.

Le routage

Défini dans **App_Start/RouteConfig.cs** : la route par défaut associe une URL à un contrôleur, une action et un paramètre optionnel.

```
routes.MapRoute(
    name: "Default",
    url: "{controller}/{action}/{id}",
     defaults: new {
        controller = "Home",
         action = "Index",
         id = UrlParameter.Optional
     }
);
```

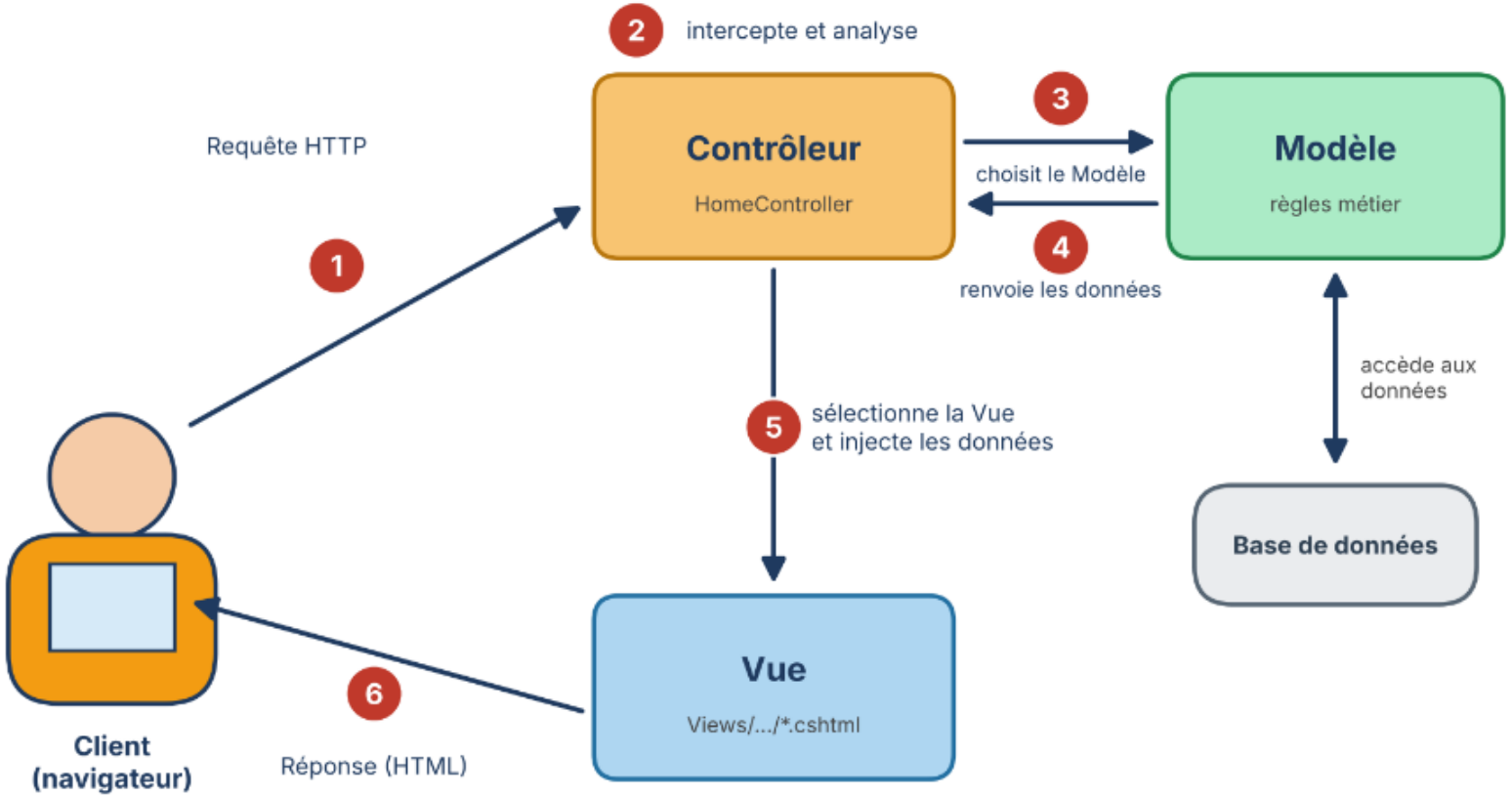
{controller}
Home → HomeController
(valeur par défaut)

{action}
Index → méthode appelée
(valeur par défaut)

{id}
paramètre optionnel

TP N°3:
Création d'une page web avec ASP.NET: Contrôleur

Rappel



Cycle d'une requête MVC

1. Le client envoie une requête depuis la Vue
2. Le Contrôleur intercepte et analyse la requête
3. Il détermine la partie du Modèle concernée
4. Le Modèle gère les données, applique les règles métier et renvoie les données au Contrôleur
5. Le Contrôleur sélectionne la Vue et lui injecte les données
6. La Vue affiche les données au client

Création d'une page web avec ASP.NET: Modèle, Vue Razor et Routage

Structure d'un projet ASP.NET MVC

Élément	Rôle
`Controllers/`	Classes contrôleur (`EtudiantsController.cs`) : reçoivent les requêtes
`Models/`	Classes métier (`Etudiant.cs`, `Prof.cs`)
`Views/`	Vues Razor `.cshtml` : un sous-dossier par contrôleur
`Views/Shared/`	Vues partagées, dont `_Layout.cshtml`
`Content/` et `Scripts/`	Fichiers statiques : CSS, JavaScript
`App_Start/RouteConfig.cs`	Déclaration des routes (URL → contrôleur/action)

Le projet cible **.NET Framework 4.8** et s'exécute sous **IIS Express** (F5 dans Visual Studio).

Création du Contrôleur

- Clic droit sur **Controllers** > **Ajouter** > **Contrôleur**.
- Sélectionner **MVC 5 Controller – Empty** puis **Ajouter**.
- Saisir le nom : **EtudiantsController** (le suffixe `Controller` est obligatoire).
- Une classe qui hérite de `Controller` (namespace `System.Web.Mvc`) est générée.

Le nom du contrôleur détermine l'URL : `EtudiantsController` répond à `/Etudiants`.

Création du Contrôleur

```
public class EtudiantsController : Controller
{
    public ActionResult Index()
    {
            // Logique métier
            return View();
    }
}
```

- **Controller** : classe C# qui pilote la logique et reçoit les requêtes.
- **ActionResult** : type de retour d'une action (vue, message, redirection…).
- **Index()** : action appelée par défaut si l'URL n'en précise pas.
- **View()** : cherche `Views/Etudiants/Index.cshtml` et lui passe les paramètres si mentionnés.

ActionResult : Exemple

```
public ActionResult Details(int id)
{
    if (id <= 0)
        return RedirectToAction("Index");   // redirection

    var etudiant = liste.FirstOrDefault(e => e.Id == id);
    if (etudiant == null)
        return HttpNotFound();              // erreur 404

    return View(etudiant);                  // la vue Details
}
```

- Une même action renvoie **des résultats différents** selon le cas : d'où `ActionResult`.

Création du Modèle

```
public class Etudiant
    {
        public int Id { get; set; }
        public string Nom { get; set; }
        public string Prenom { get; set; }
        public int Age { get; set; }
    }
```

- Un **modèle** est une classe qui représente une entité métier.
- Il se place dans le dossier **Models** ; ses **propriétés** (`{ get; set; }`) décrivent les données.
- Il transporte les données entre le contrôleur et la vue.

Création d'une Vue

- Clic droit sur la méthode **Index** puis **Ajouter une vue**.
- Sélectionner **Vue MVC 5**, puis **Empty (sans modèle)** et cliquer sur **Ajouter**.
- Le fichier `Views/Etudiants/Index.cshtml` est créé.
- Convention : **Views / nom du contrôleur (sans Controller) / nom de l'action .cshtml**



Qu'est-ce que Razor ?

- **Razor** est le moteur de vues d'ASP.NET : un fichier `.cshtml` mélange **HTML** et **C#**.
- Le caractère **@** fait passer du HTML au C#.
- Razor détecte seul la fin de l'expression C#.

Dans le fichier .cshtml

```
<p>Bonjour @nom</p>
```

HTML reçu par le navigateur (si nom = « Sara »)

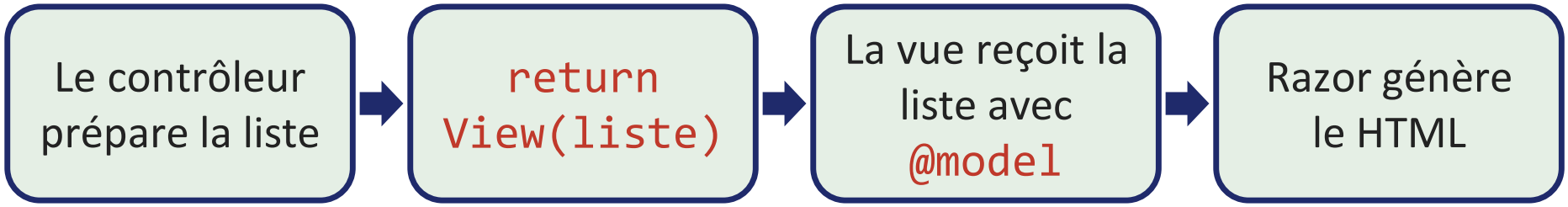
```
<p>Bonjour Sara</p>
```

Création d'une Vue : Index.cshtml

```
@model List<ApplicationWeb.Models.Etudiant>
<h2>Liste des étudiants</h2>
<table border="1" cellpadding="5">
    <thead>
<tr><th>Nom</th><th>Prénom</th><th>Âge</th><th>Détail</th></tr>
    </thead>
    <tbody>
 @foreach (var e in Model)
    { <tr>
        <td>@e.Nom</td>
        <td>@e.Prenom</td>
        <td>@e.Age</td>
      </tr>
    }
    </tbody>
</table>
```

Élément Razor	Rôle
@model	Indique le type de données reçu
@Model	Accède aux données dans la vue
@foreach (...)	Boucle C# dans la vue
@e.Propriété	Affiche les données du modèle

Du Modèle à la Vue

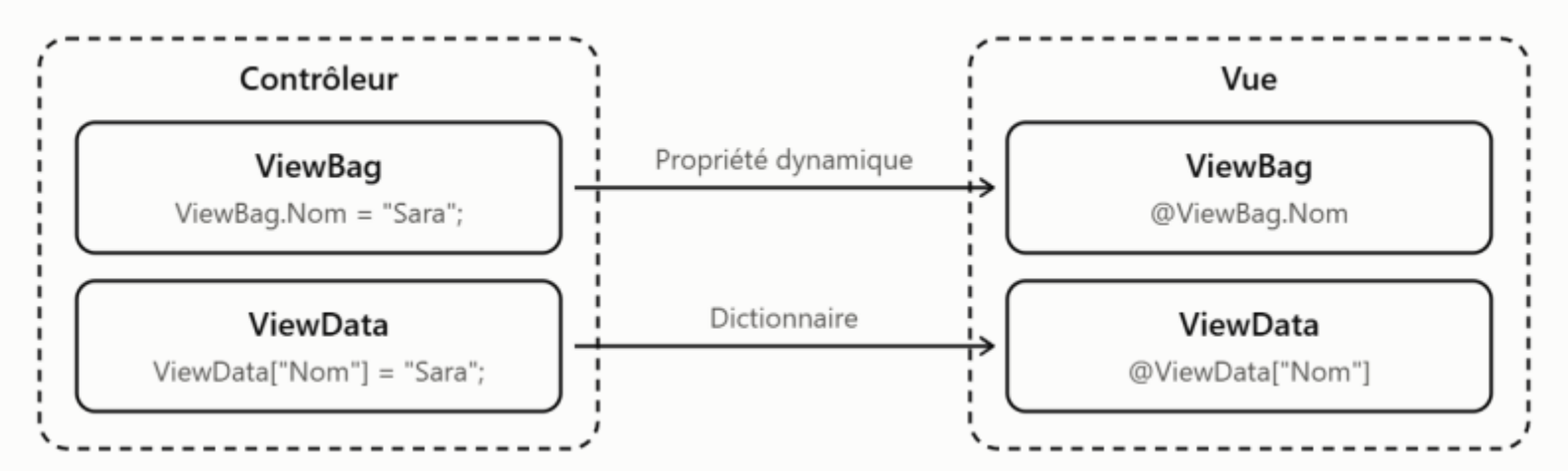


- Le contrôleur **prépare** les données (le modèle) ; la vue ne fait que les **afficher**.
- Le type déclaré par `@model` doit correspondre à l'objet passé à `View(...)`.
- Le navigateur ne reçoit que du **HTML** : le code C# est exécuté sur le serveur.

Transfert de données vers la Vue

Pour Transfert des données: Vue :

- **ViewBag, ViewData:** permet de stocker des données dans une action de contrôleur qui sera utilisée dans la vue correspondante.



Transfert de données vers la Vue

Contrôleur

```
public ActionResult Index(int? id)
{
    ViewData["id"] = id;
    return View();
}
```

Vue Index.cshtml

```
<p><b>Hello @ViewData["id"]</b></p>
```

- `/Etudiants/Index/5` affiche **Hello 5** : le segment `5` de l'URL est lié au paramètre `id`.
- `int?` rend le paramètre facultatif : sans valeur, `id` vaut `null`.

Transfert de données vers la Vue

Contrôleur

```
public ActionResult Index()
{   ViewBag.Title = "Liste des étudiants";
    ViewBag.Message = "Bienvenue sur le site";
    return View();
}
```

Vue Index.cshtml

```
<h2>@ViewBag.Title</h2>
<p>@ViewBag.Message</p>
```

- L'utilisation : `ViewBag.` suivi du **nom** que l'on choisit.
- Le nom doit être **identique** (même casse) dans le contrôleur et dans la vue.

Le routage en ASP.NET

- Le **routage** détermine comment une **URL entrante** est associée à un **contrôleur** et une **action**.

```
routes.MapRoute(
    name: "Default",
    url: "{controller}/{action}/{id}",
    defaults: new { controller = "Etudiants", action = "Index",
                    id = UrlParameter.Optional });
```

Élément	Signification
`{controller}/{action}/{id}`	Structure des URL MVC
`controller = "Etudiants"`	Contrôleur par défaut si l'URL est vide
`action = "Index"`	Action par défaut
`UrlParameter.Optional`	Paramètre `id` **facultatif**

TP N° 4